

ARTIFICIAL NEURAL NETWORKS

MID-SEM QUICK REVISION NOTES

Units 1–5 • Definitions • Key formulas • Intuition • Diagrams

1. 1-Minute Formula Sheet

Topic	Formula / fact
Linear neuron	$z = \sum w_i x_i + b$
Linear regression	$\hat{y} = w^T x + b$
MSE	$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$
Gradient descent	$\theta \leftarrow \theta - \eta \nabla J(\theta)$
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$, $\sigma'(z) = \sigma(z)[1-\sigma(z)]$
tanh	$\tanh'(z) = 1 - \tanh^2(z)$
ReLU	$\max(0, z)$, derivative: 1 if $z > 0$ else 0
L1 penalty	$\lambda \sum w_i $
L2 penalty	$\lambda \sum w_i^2$
Regularized cost	$J = \text{data loss} + \text{penalty}$
Neuron decision	$\text{step}(w^T x + b)$: output 1 if $z \geq 0$, else 0
Backprop idea	forward pass $\rightarrow$ loss $\rightarrow$ chain rule gradients $\rightarrow$ update weights

UNIT 1 — Neuron & Basic Neural Networks

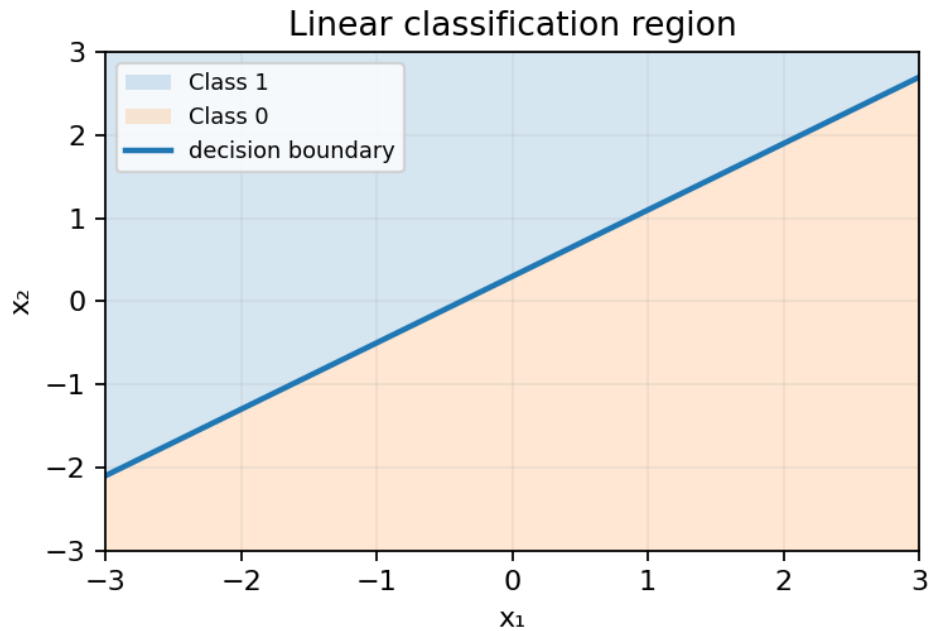
Linear Regression

- Predicts a continuous numerical value using a linear relationship between inputs and output.
- Example: house price = $w_1(\text{area}) + w_2(\text{rooms}) + b$.
- A single linear neuron can represent the same equation.

$$\hat{y} = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = w^T x + b$$

Classification Regions

- Classification divides the input space into regions belonging to different classes.
- A single neuron with a step/sign decision creates a straight-line (2D), plane (3D), or hyperplane decision boundary.
- Points on one side $\rightarrow$ one class; the other side $\rightarrow$ another class.



Neural Network as a Collection of Lines

- Each linear neuron computes a weighted sum, so its decision boundary is a line/hyperplane.
- Multiple neurons can create multiple boundaries; combining them can form more complex regions.
- This is the geometric view of neural networks.

Matrix Multiplication

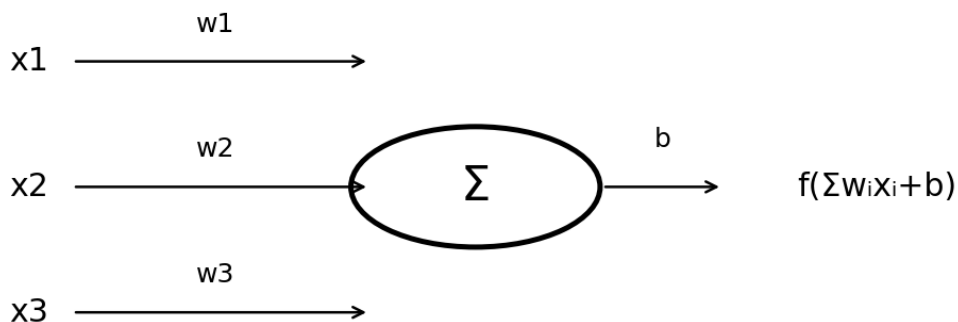
- Vector/matrix form makes many neuron calculations compact and efficient.
 - For one layer: $z = Wx + b$.
 - For a batch of examples, matrix multiplication computes many neurons/examples together.
- If W is $m \times n$ and x is $n \times 1$, then Wx is $m \times 1$.

Biological / Human Neuron

- Dendrites receive signals; the cell body combines them; the axon sends the output.
- A neuron fires when the combined input reaches sufficient activation.

Artificial Neuron

- Mathematical model inspired by a biological neuron.
- Inputs are multiplied by weights, added together with bias, then passed through an activation function.



Inputs and Weights

- x_i = input feature/signal.
- w_i = importance/sign of that input.
- Large positive weight strengthens the input; negative weight suppresses/reverses it.

Weighted Summation

- Multiply each input by its weight and add them.
- Bias is added before activation.

$$z = \sum w_i x_i + b$$

Threshold

- A threshold decides whether a neuron activates.
- For a binary step neuron: output 1 when $z \geq 0$, otherwise 0.
- Threshold can be written as a separate value θ : output 1 if $\sum w_i x_i \geq \theta$.

Bias

- Bias shifts the decision boundary left/right (or generally changes its position).
- Without bias, a linear decision boundary is forced to pass through the origin.

$$z = w^T x + b$$

Boolean Gates — AND, OR, NOT

- Boolean gates take binary inputs (0/1) and produce a binary output.
- AND $\rightarrow$ 1 only when both inputs are 1.
- OR $\rightarrow$ 1 when at least one input is 1.
- NOT $\rightarrow$ flips the input: $0 \rightarrow 1$ and $1 \rightarrow 0$.
- A single threshold neuron can implement AND, OR and NOT because their classes are linearly separable.

x_1	x_2	AND	OR	NOT (x_1)
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Basic Network Representation

- Input layer receives features.
- Connections carry weights.
- Neurons calculate weighted sums + bias and activation.
- Output layer produces prediction/class.

Basic Classification Using Neurons

- Compute $z = w^T x + b$.
- Apply activation/threshold.
- The sign of z determines which side of the decision boundary the point lies on.

UNIT 2 — Error, Cost & Optimization

MSE — Mean Squared Error

- Measures average squared difference between true values and predictions.
- Squaring makes all errors positive and penalizes large errors more strongly.
- Lower MSE means predictions are closer to targets.

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$$

Cost Function

- A single number that tells how bad the model's predictions are.
- Training tries to minimize the cost.
- For regression, MSE is a common cost/loss choice.

Error Minimization

- Goal: find weights/biases that make the loss as small as possible.
- Visualize cost as a surface: training moves toward a low point/minimum.

Basic Parameter Optimization

- Parameters = weights and biases.
- Gradient tells the direction of greatest increase in cost.
- Move in the opposite direction to reduce cost.

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta(\partial J / \partial \theta) \quad ; \quad \eta = \text{learning rate}$$

Basic Error-Related Learning Concept

- Prediction error generates a learning signal.
- Gradient-based learning changes parameters so future predictions reduce the error.
- Learning rate controls step size: too large may overshoot; too small makes learning slow.

Regularization / Penalty Concept

- Regularization discourages overly large/complex weights.
- It adds a penalty term to the original loss.
- Purpose: reduce overfitting and improve generalization.

$$\text{Regularized cost} = \text{data loss} + \lambda \times \text{penalty}$$

UNIT 3 — Regression, Classification & Multilayer Networks

Linear Regression

- Output is continuous.
- Model is a weighted sum of input features plus bias.
- Typically trained by minimizing MSE.

$$\hat{y} = w^T x + b$$

MSE-Based Optimization

- Compute predictions → calculate MSE → calculate gradients → update w and b → repeat.
- The process is gradient descent when updates use the negative gradient.

Linear Classification

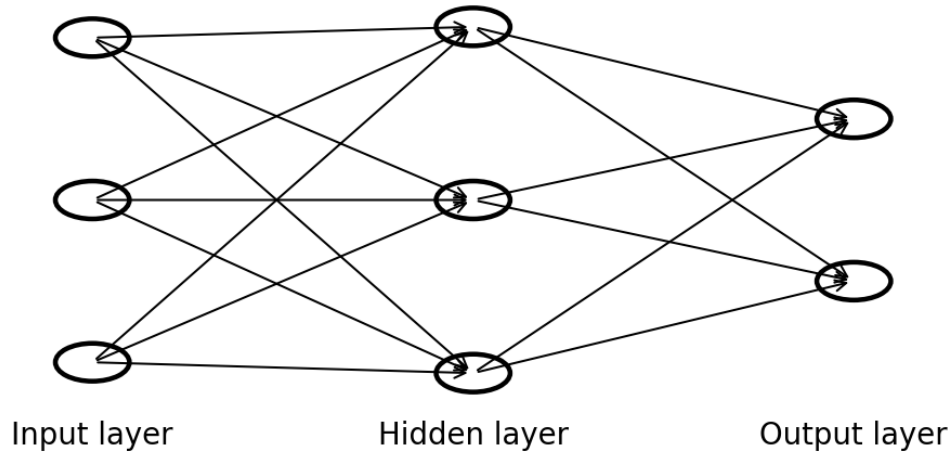
- Used for class labels such as 0/1.
- A linear classifier uses $z = w^T x + b$ and a threshold/appropriate activation.
- Its decision boundary is $w^T x + b = 0$.

Classification Regions

- The boundary separates input space into class regions.
- One linear neuron can create only one hyperplane boundary.
- Non-linear problems such as XOR need multiple boundaries/layers.

Basic Multilayer Network

- A multilayer neural network contains input, one or more hidden layers, and an output layer.
- Hidden layers transform the input into increasingly useful representations.
- Nonlinear activation functions let the network represent nonlinear boundaries.



Input Layer

- Receives the feature values; usually does not perform a learned transformation itself.

Hidden Layer

- Intermediate neurons that learn combinations/features from previous layer.
- Multiple hidden layers form a deep neural network.

Output Layer

- Produces final prediction.
- Typical binary classification: one output neuron with sigmoid.
- Regression: often one linear output neuron.

UNIT 4 — Backpropagation, XOR & Activations

Backpropagation Algorithm

- Efficient method for computing gradients of the loss with respect to every weight/bias.
- Steps: 1) forward pass, 2) calculate loss, 3) propagate error backward using chain rule, 4) update parameters.
- Backpropagation computes gradients; gradient descent uses those gradients to update parameters.

XOR Problem

- XOR output is 1 when inputs are different and 0 when they are the same.
- Its positive and negative examples cannot be separated by one straight line.
- Therefore a single perceptron/linear neuron cannot solve XOR.
- A multilayer network with nonlinear activation can solve it.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Boolean-Function Implementation

- A truth table defines desired output for every input combination.
- Design neurons/gates that reproduce those mappings.
- AND/OR/NOT are linearly separable; XOR requires a combination of intermediate decisions.

Truth-Table Based Neural Network

- List all input combinations and desired outputs.
- Choose neuron weights/biases so the activation matches each row.
- For multi-step Boolean functions, use hidden neurons to implement intermediate functions.

AND / OR / NOT Implementation

- Use a binary step activation.
- AND example: $z = x_1 + x_2 - 1.5 \rightarrow \text{step}(z)$.
- OR example: $z = x_1 + x_2 - 0.5 \rightarrow \text{step}(z)$.
- NOT example: $z = -x + 0.5 \rightarrow \text{step}(z)$.

Decision / Classification Regions

- A neuron creates a boundary; a network combines boundaries to create complex shapes.
- The region where the output is 1 is the positive/class-1 region.

Polygon Representation

- Multiple linear boundaries can enclose a polygonal classification region.
- Each neuron can represent one half-plane; ANDing conditions keeps only their intersection.

Triangle Representation

- Three line boundaries can form a triangular region.
- A hidden neuron can test each half-plane; an output neuron can combine them.

Square Representation

- Four line boundaries can form a square/rectangle.
- Each boundary corresponds to a linear inequality.

Combination of Polygons

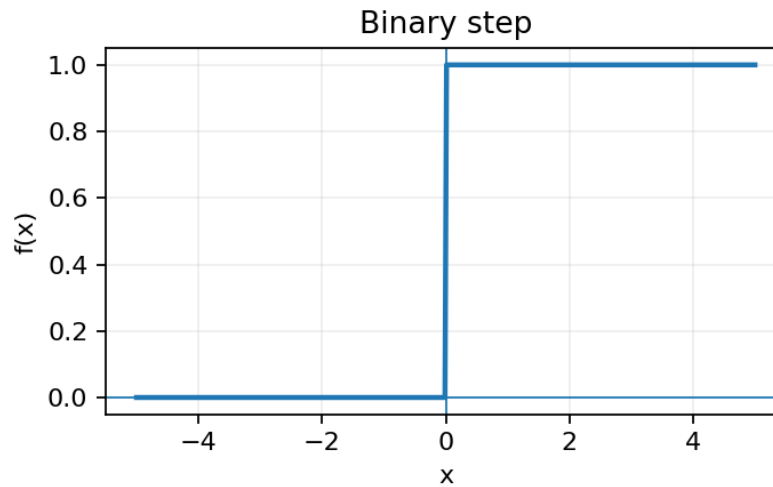
- OR-like combinations can create unions of regions.
- AND-like combinations can create intersections.
- This gives multilayer networks their geometric flexibility.

Activation Functions

Binary Step

- Hard threshold; useful for simple Boolean/perceptron logic.
- Not differentiable at 0 and derivative is 0 almost everywhere, so it is unsuitable for standard gradient-based deep training.

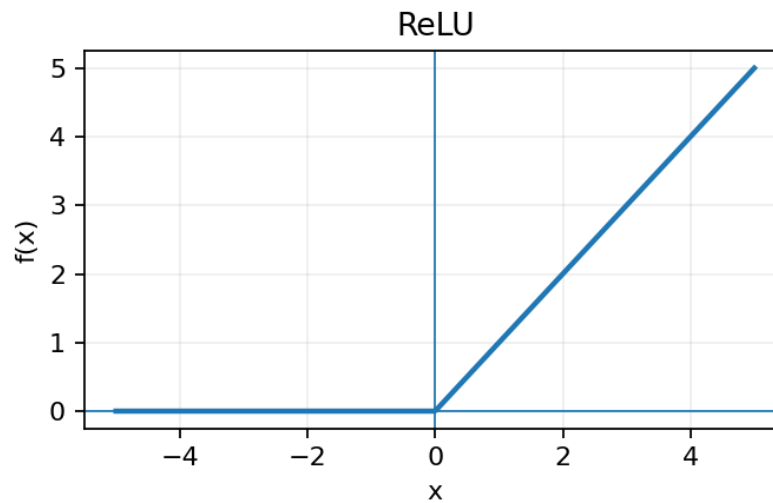
$$f(z) = 1 \text{ if } z \geq 0 \text{ else } 0$$



ReLU

- Keeps positive values and converts negative values to 0.
- Fast and common in hidden layers.
- Can suffer from 'dying ReLU' when neurons remain on the negative side.

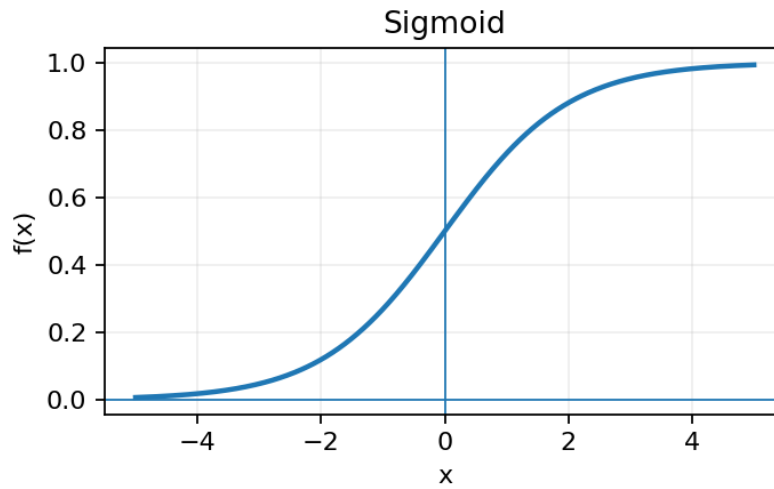
$$\text{ReLU}(z) = \max(0, z)$$



Sigmoid

- Maps any real number to (0,1).
- Useful for binary-class probability-like outputs.
- Can have very small gradients for large positive/negative z (vanishing gradient).

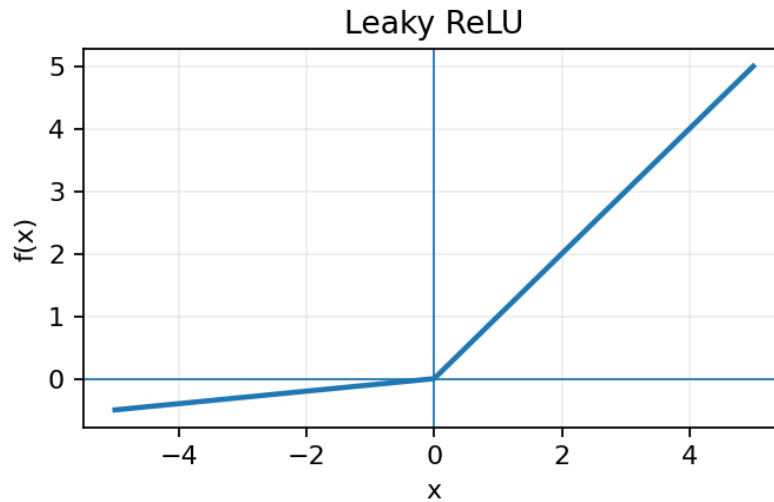
$$\sigma(z) = 1 / (1 + e^{-z}), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$



Leaky ReLU

- Like ReLU, but gives a small negative slope instead of zero for negative inputs.
- Helps reduce dead neurons.

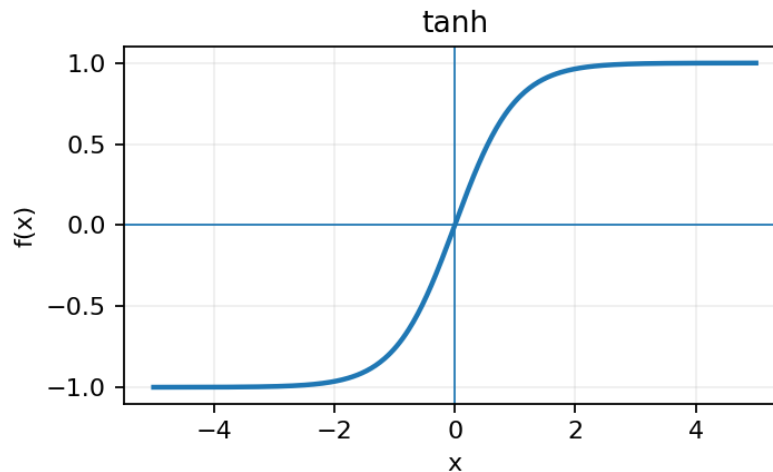
$f(z) = z$ if $z \geq 0$; αz if $z < 0$, usually small α



tanh

- Maps values to $(-1, 1)$.
- Zero-centered, unlike sigmoid.
- Can still suffer from vanishing gradients for large $|z|$.

$\tanh'(z) = 1 - \tanh^2(z)$



Basic Multilayer Network Training

- Initialize weights → forward pass → calculate loss → backpropagate gradients → update weights → repeat for many iterations/epochs.
- Training is complete when loss is sufficiently low and validation performance is satisfactory.

UNIT 5 — Detailed Backpropagation & Regularization

Backpropagation

- Uses the chain rule to determine how much each parameter contributed to the final error.
- Start from output error and move backward layer by layer.

For a parameter w : $\partial L / \partial w = (\partial L / \partial \text{output}) \times (\partial \text{output} / \partial z) \times (\partial z / \partial w)$

Differentiation

- Derivative tells how much a quantity changes when its input changes.
- In neural networks, derivatives provide the direction and magnitude needed for parameter updates.

Chain Rule

- Used when one function depends on another function.
- Multiply local derivatives along the path from a parameter to the loss.

If $y = f(g(x))$, then $dy/dx = (dy/dg)(dg/dx)$

Sigmoid Differentiation

- The sigmoid derivative can be written using the sigmoid output itself.
- This makes backpropagation computationally convenient.

$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$

Error Calculation

- Compare prediction $\hat{y}$ with target y using the chosen loss.
- For MSE, the error is squared difference; for many classification models, cross-entropy is commonly used.

MSE sample loss: $L = \frac{1}{2}(y - \hat{y})^2 \rightarrow \partial L / \partial \hat{y} = \hat{y} - y$

Output-Layer Derivatives

- For an output neuron, compute loss derivative with respect to output.
- Multiply by activation derivative to obtain derivative with respect to pre-activation z .
- Then propagate to weights and inputs.

Hidden-Layer Derivatives

- A hidden neuron affects the loss indirectly through the next layer.
- Its gradient is the downstream gradient multiplied by its local activation derivative.
- This is why chain rule is central to backpropagation.

Weight Derivatives

- For $z = w^T x + b$, $\partial z / \partial w = x$.
- Therefore weight gradient is closely related to input activation $\times$ downstream error.
 $\partial L / \partial w = \delta x$ and $\partial L / \partial b = \delta$

Weight Update

- Move weights opposite to the gradient.
- Repeat over training examples/batches.
 $w \leftarrow w - \eta (\partial L / \partial w)$

Matrix-Based Backpropagation Calculations

- For layer l : $z^l = W^l a^{l-1} + b^l$ and $a^l = f(z^l)$.
- Output/hidden error vector: $\delta^l = (W^{l+1})^T \delta^{l+1} \odot f'(z^l)$.
- Gradients: $\partial L / \partial W^l = \delta^l (a^{l-1})^T$ and $\partial L / \partial b^l = \delta^l$.
- $\odot$ means element-wise multiplication.

Core forward equations: $z^1 = W^1 a^{1-1} + b^1$; $a^1 = f(z^1)$

Overfitting

- Model learns training data too closely, including noise or accidental patterns.
- Training error becomes low while performance on unseen data is worse.
- Common controls: more data, simpler model, early stopping, dropout, and regularization.

Training vs Testing Performance

- Training set: used to learn parameters.
- Validation set: used for tuning/model selection.
- Test set: final unbiased estimate on unseen data.
- Typical overfitting pattern: training loss $\downarrow$ while validation/test loss eventually $\uparrow$.

L1 Regularization

- Adds the absolute values of weights to the loss.
- Encourages sparse solutions; some weights may become exactly zero.
 $J_{\text{regularized}} = J_{\text{data}} + \lambda \sum |w_i|$

L2 Regularization

- Adds squared weights to the loss.
- Encourages small weights and generally spreads information across parameters rather than forcing exact zeros.
 $J_{\text{regularized}} = J_{\text{data}} + \lambda \sum w_i^2$

Penalty Function

- Penalty is the extra term added to discourage model complexity.
- λ controls strength: larger $\lambda \rightarrow$ stronger pressure toward smaller weights.
- L1 $\rightarrow$ absolute-value penalty; L2 $\rightarrow$ squared-weight penalty.

2. Exam-Day Checklist

- Know neuron equation: $z = w^T x + b$ and output $a = f(z)$.
- Know why bias is needed: it shifts the decision boundary.
- Know why one linear neuron cannot solve XOR.

- Know the roles of input, hidden and output layers.
- Know MSE and gradient-descent update.
- Know all five activations: step, ReLU, sigmoid, leaky ReLU, tanh.
- Know backprop order: forward $\rightarrow$ loss $\rightarrow$ chain rule $\rightarrow$ gradients $\rightarrow$ update.
- Know sigmoid derivative: $\sigma(1-\sigma)$.
- Know matrix backprop: $\delta^l = (W^{l+1})^T \delta^{l+1} \odot f'(z^l)$.
- Know overfitting signs and L1 vs L2 difference.
- For Boolean questions, start from the truth table and construct threshold conditions.
- For geometry questions, remember: one neuron = one half-plane; multiple neurons = combined regions/polygons.